ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

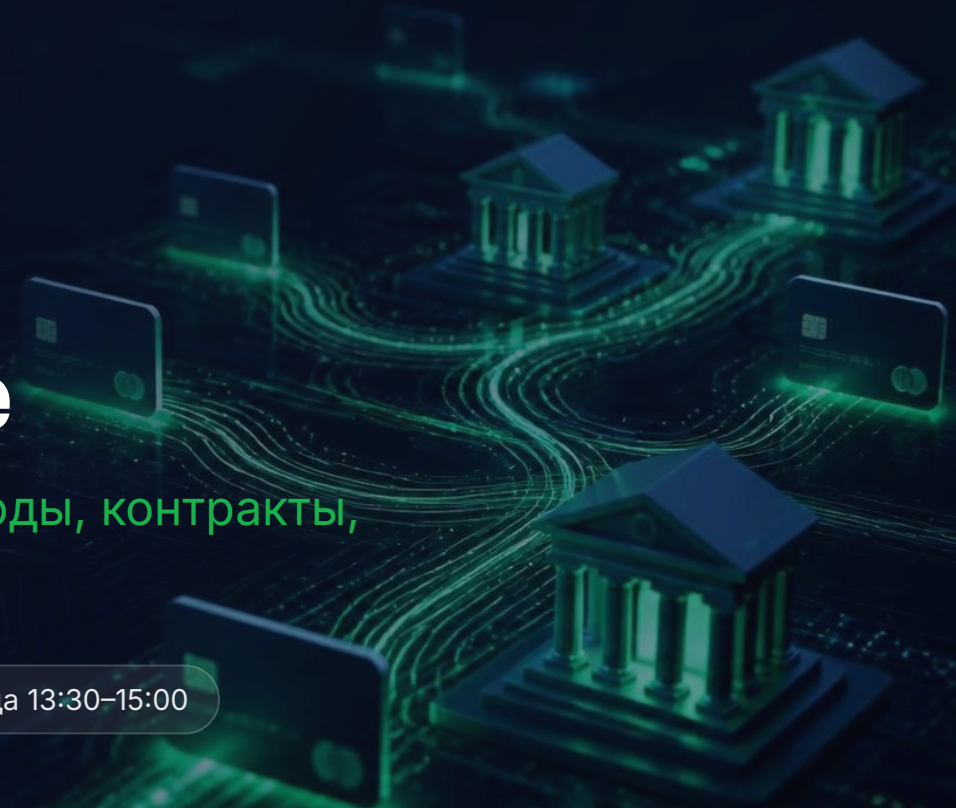
API-тестирование

Модуль 5 · Лекция 9 · REST, статус-коды, контракты, OpenAPI

Вера Терентьева · Lekton

4 курс

28 октября · Среда 13:30–15:00



МОДУЛЬ 5 · ЛЕКЦИЯ 9

План лекции

01 Место API-тестов в пирамиде тестирования

02 HTTP как язык описания API: ресурсы, методы, статус-коды

03 То, что обычно пропускают: заголовки и ловушки данных

04 Контракт как артефакт и OpenAPI как машиночитаемая форма

05 Инструменты: REST-assured для регрессии, Postman для разведки

06 Практика на СМП: базовый и негативный тест на авторизацию

07 Организация тест-сьюта: спецификации, данные, группировка

08 Интерактивы: ошибка в тесте и расхождение спецификации с кодом

МОДУЛЬ 5 · ЛЕКЦИЯ 9

API-тесты — уровень, где ломается интеграция, а не логика

> API-тест проверяет не «правильно ли написана функция», а «правильно ли сервис отвечает на запрос по контракту».

- Курс «Тестирование программного обеспечения», модуль 5 — API-тестирование REST-сервисов.
- Лекция 9 закладывает фундамент: язык HTTP, контракт, спецификация и инструменты.
- Сегодня вы научитесь читать спецификацию как тестировщик и писать первый осмысленный API-тест.
- Лекция 10 добавит паттерны: сценарии, доступ, идемпотентность, асинхронность.
- Практика модуля — артефакт 4: REST-тесты и проверка цепочки сервисов.

Модуль 5 — это две лекции и один артефакт

> API-тестирование в курсе — не отдельная тема, а уровень пирамиды, который вы будете наполнять до конца семестра.

- Лекция 9: язык HTTP, контракт, OpenAPI, REST-assured и Postman.
- Лекция 10: паттерны — позитив и негатив, аутентификация, идемпотентность, асинхронные контракты.
- Артефакт 4: покрыть эндпоинты целевых сервисов, все decline-коды, проверить цепочку сервисов.
- Объект тестирования — СМП: одиннадцать сервисов, PostgreSQL, RabbitMQ, единая точка входа Gateway.
- Дальше модуль 6 углубит интеграции: БД в контейнерах, WireMock, Testcontainers.

API — это уровень, где видно, что сервисы не договорились

> API-тесты дороже unit-тестов, но именно они ловят класс дефектов, невидимый на уровне отдельного класса.

- Unit проверяет логику в изоляции, E2E — всю систему целиком, API — стык между сервисами.
- API-тест в десятки раз быстрее E2E и в разы стабильнее, потому что не зависит от интерфейса.
- Он проверяет то, что реально видят другие команды: формат тела, статус-код, заголовки.
- В микросервисной архитектуре это основной уровень автоматизации, а не вспомогательный.
- Каждый контракт «поставщик — потребитель» в СМП требует отдельного покрытия.

МОДУЛЬ 5 · ЛЕКЦИЯ 9

HTTP как язык описания API

REST — это соглашение об адресах и операциях, а не протокол

> REST API — это набор ресурсов, адресуемых URL, над которыми выполняются операции, выраженные HTTP-методами.

- REST описан Роем Филдингом в 2000 году как архитектурный стиль, а не как стандарт.
- Ресурс адресуется URL: `/api/cards/4000001234560001` — это конкретная карта, а не действие.
- Операция задаётся методом: `GET` читает, `POST` создаёт или выполняет действие.
- Состояние клиента не хранится на сервере — каждый запрос самодостаточен.
- В СМП: `GET /api/cards/{pan}` — ресурс «карта», операция «прочитать».

МОДУЛЬ 5 · ЛЕКЦИЯ 9

Метод — это часть контракта, а не украшение URL

> Семантика HTTP-метода определяет, что произойдёт при повторной отправке запроса, и это напрямую влияет на тест-дизайн.

- `GET` читает ресурс и не должен иметь побочных эффектов.
- `POST` создаёт ресурс или выполняет действие и не идемпотентен по умолчанию.
- `PUT` заменяет ресурс целиком, `PATCH` обновляет частично, `DELETE` удаляет.
- `GET`, `PUT`, `DELETE` идемпотентны: повтор даёт тот же результат.
- В СМП реально используются `GET` и `POST`: чтение карт и поиск, создание карт и авторизация.

Статус-код — это часть контракта, и его тестируют отдельно от тела

> Класс статус-кода говорит, кто виноват в проблеме — клиент или сервер, — и это различение является базовым навыком API-тестирования.

- `2xx` — успех: `200 OK` для чтения и действий, `201 Created` для создания ресурса.
- `4xx` — ошибка клиента: запрос некорректен, повторять его бессмысленно.
- `5xx` — ошибка сервера: проблема на нашей стороне, и это всегда повод для разбора.
- В СМП используются `200`, `201`, `400`, `403`, `404`, `405`, `409`, `422`, `429`, `503`.
- Правильный статус-код — половина правильного поведения; тело ответа — вторая половина.

Заголовки — слой, который тестируют реже всего и который ломается чаще всего

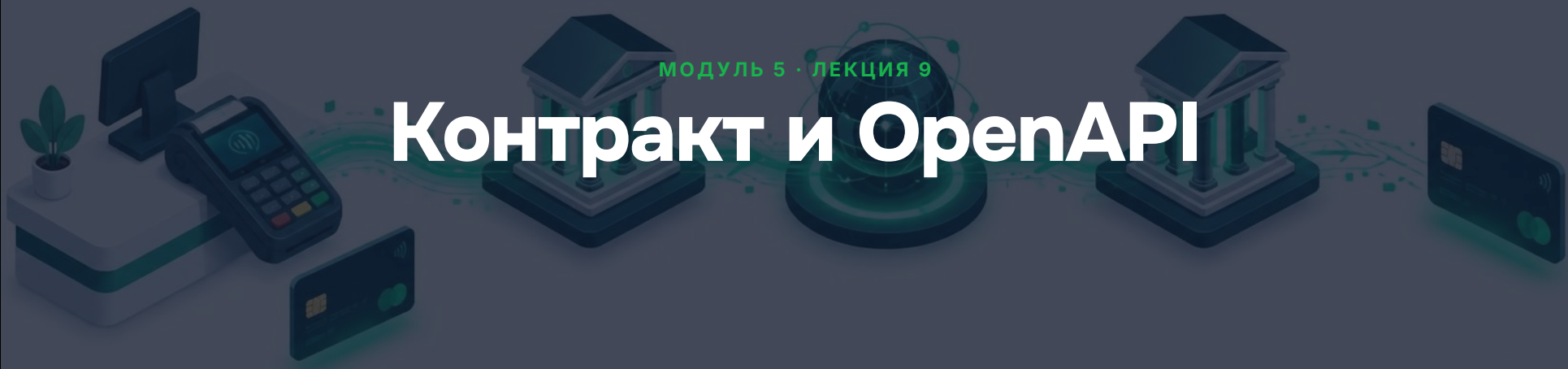
> Заголовки несут часть контракта, которой нет в теле ответа, и без их проверки тест остаётся неполным.

- `Content-Type` в запросе сообщает формат тела; без него сервер может ответить `415`.
- `Accept` в запросе сообщает, какой формат ответа клиент готов принять.
- `Retry-After` сопровождает `429` и говорит, через сколько повторять запрос.
- `X-Cache` со значениями `HIT` или `MISS` показывает, отдал ли Gateway ответ из кеша.
- `Content-Disposition` в CSV-экспорте сообщает имя файла — часть контракта выгрузки.

Большинство ошибок в API-тестах — это ошибки в данных, а не в запросах

> Прежде чем писать проверки, нужно понять, как устроены данные: единицы измерения, типы, отсутствие значений и форматы даты.

- Все суммы в СМП — в минорных единицах: `150000` означает 1500 рублей, а не 150 тысяч.
- `amount` — число, `currencyCode` и `mcc` — строки; путаница типов даёт `400`.
- Отсутствующее поле и поле со значением `null` — разные вещи, и `nullValue()` их различает.
- Даты — ISO 8601 со смещением: `2026-06-01T10:30:00Z`; формат без `Z` может не разобраться.
- Спецификация СМП терпима к лишним полям, но контракт описывает только ожидаемые.



МОДУЛЬ 5 · ЛЕКЦИЯ 9

Контракт и OpenAPI

Контракт — это артефакт, а тест — проверка соответствия этому артефакту

> Пока контракт не зафиксирован письменно, у команд нет общего представления о том, что считается правильным поведением.

- Контракт определяет эндпоинты, методы, параметры, тела, статус-коды и заголовки.
- Владелец контракта — команда-поставщик, но соблюдение проверяют обе стороны.
- Контракт живёт в репозитории рядом с кодом и версионируется вместе с ним.
- Без письменного контракта договорённость существует только в головах и расходится за месяц.
- В СМП контракт зафиксирован в `docs/api-spec.md` и в аннотациях OpenAPI в коде.

МОДУЛЬ 5 · ЛЕКЦИЯ 9

OpenAPI — это контракт, который умеет читать машина

> OpenAPI превращает договорённость в машиночитаемый документ, из которого можно одновременно получить документацию, валидацию и генерацию кода.

- OpenAPI — индустриальный стандарт описания REST API, ранее известный как Swagger.
- Формат — YAML или JSON; описывает эндпоинты, схемы, статус-коды и примеры.
- Один документ даёт три вещи: документацию, интерактивную справку, основу для генерации.
- В СМП каждый сервис публикует свой документ, а Gateway собирает их в единый Swagger UI на `/docs`.
- Спецификация и код могут расходиться — и тогда тест должен это поймать.

МОДУЛЬ 5 · ЛЕКЦИЯ 9

Спецификация СМП: что смотреть в первую очередь

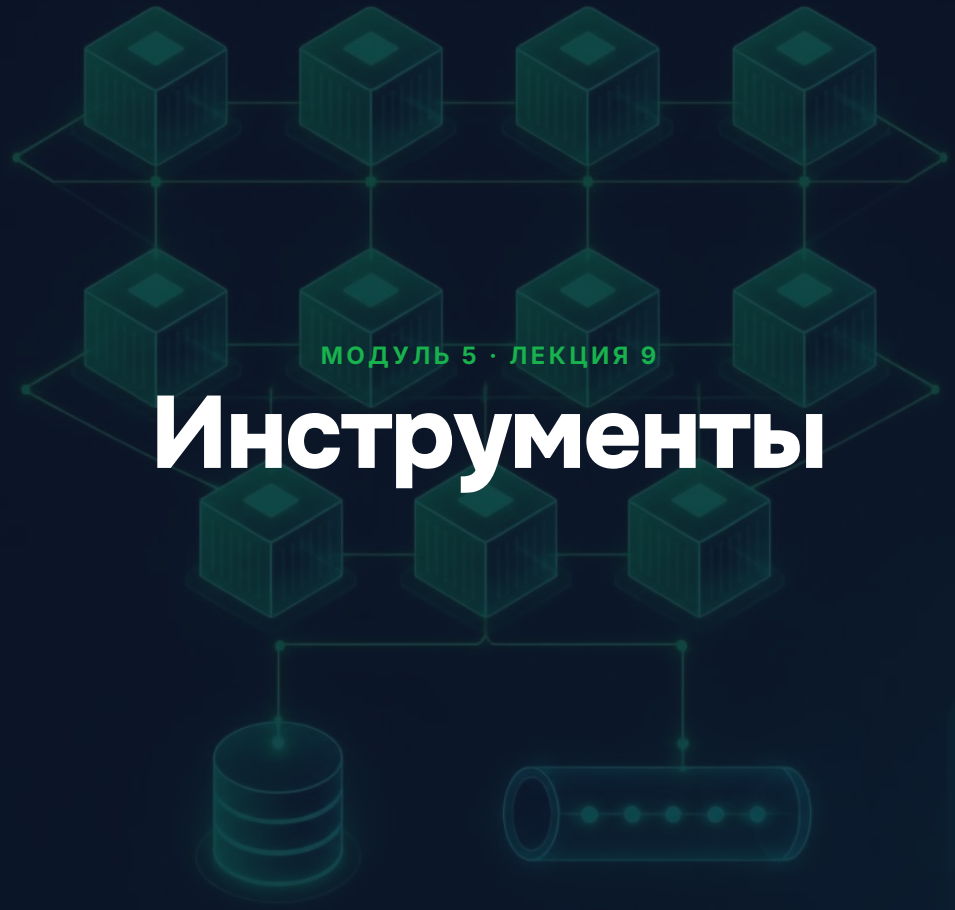
> Спецификацию читают не целиком, а по четырём слоям: разделы, эндпоинт, модель, коды.

- Документ разбит по доменам: Health, Cards, Transactions, Simulator, Dashboard, WebSocket, Internal.
- Для эндпоинта смотрят метод, путь, параметры, тело, успешный ответ и список ошибок.
- Модели описаны таблицами полей: `Card`, `AuthorizationRequest`, `AuthorizationResponse`, `Transaction`.
- Коды ответов — двухуровневые: HTTP-статус и доменный `responseCode` по ISO 8583.
- Внешний путь и внутренний путь различаются: Gateway переписывает `/api/transactions` в `/api/internal/route`.

Контракт проверяют двумя способами: схемой и договором с потребителем

Проверка схемы ловит нарушения формата внутри одного сервиса, consumer-driven контракты ловят нарушения договорённости между командами.

- Проверка схемы: ответ валидируется против JSON-схемы из OpenAPI — дёшево и сразу покрывает все поля.
- В REST-assured это одна строка с `matchesJsonSchemaInClasspath`.
- Consumer-driven контракт: потребитель фиксирует, что ему нужно, поставщик проверяет себя против этого.
- Pact — инструмент consumer-driven подхода; в курсе он не обязателен, но идею надо знать.
- Схема отвечает «формат правильный», договор отвечает «меня не сломали».



МОДУЛЬ 5 · ЛЕКЦИЯ 9

Инструменты

REST-assured: тест читается как тест-кейс

> Сила REST-assured не в том, что он «умеет HTTP», а в том, что структура теста совпадает со структурой ручного тест-кейса.

- Библиотека для Java с DSL в стиле given / when / then.
- `given()` — предусловия: адрес, заголовки, параметры, тело запроса.
- `when()` — действие: метод и путь.
- `then()` — проверки: статус, тело через JSONPath и Hamcrest, заголовки, время.
- GPath-выражения позволяют фильтровать массивы прямо в проверке: `transactions.findAll { it.id == ... }`.

Postman: инструмент разведки, а не регрессии

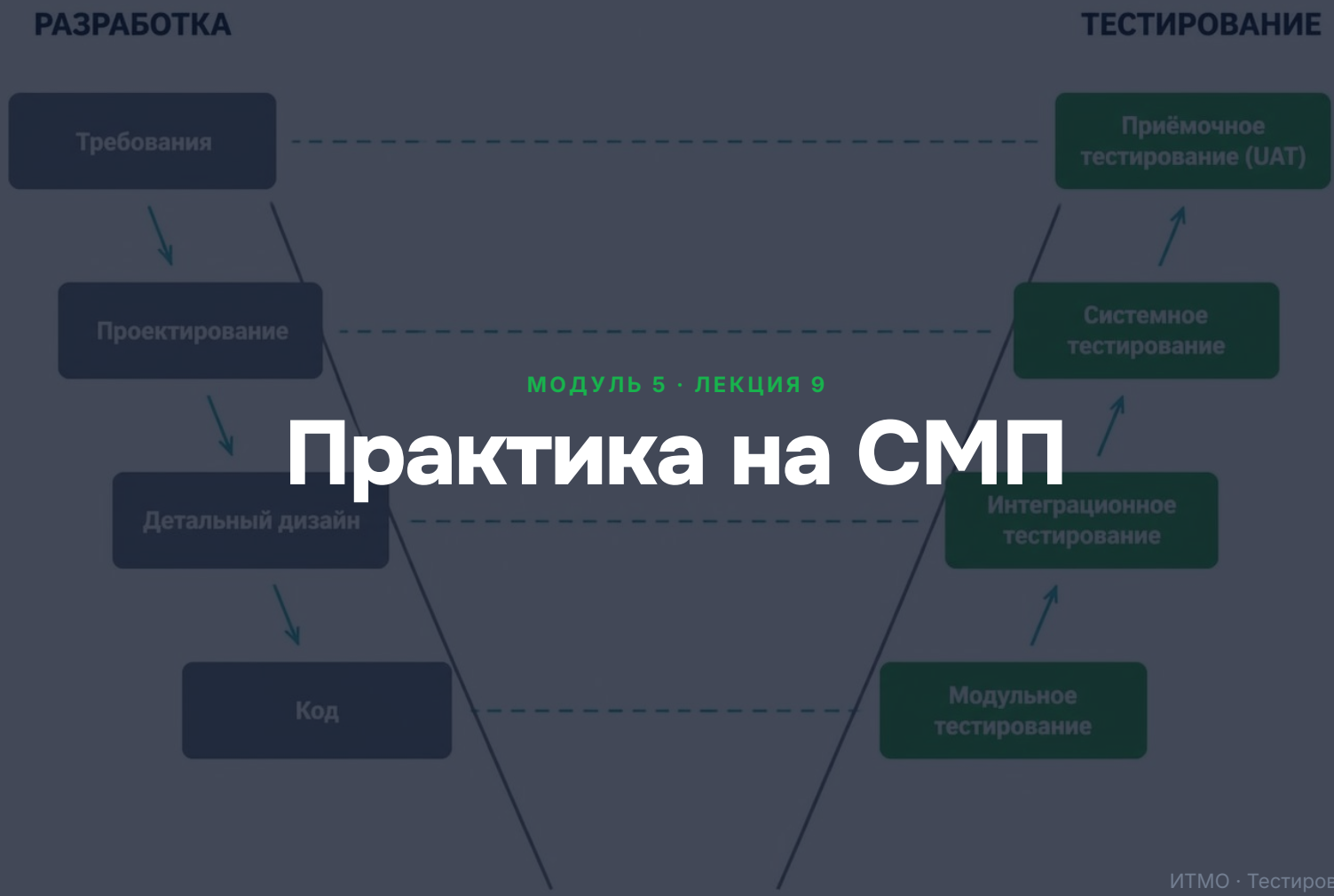
> Postman незаменим на этапе, когда вы ещё не знаете, как ведёт себя эндпоинт, и вреден как единственный инструмент регресса.

- Графический клиент: собрал запрос, нажал Send, увидел ответ — без компиляции и запуска.
- Коллекции группируют запросы по сценариям, окружения переключают адреса одним кликом.
- Переменные позволяют взять значение из ответа и подставить в следующий запрос.
- Применение: разведка нового эндпоинта, отладка дефекта, разовая демонстрация.
- Ограничение: коллекцию сложно версионировать, ревьюить и запускать в CI.

Критерий выбора: нужна ли эта проверка завтра

> Инструмент выбирается не по вкусу, а по сроку жизни проверки.

- Postman — проверка живёт один раз: разведка, демонстрация, отладка чужого сервиса.
- REST-assured — проверка живёт в регрессии и должна запускаться при каждом изменении.
- Правильный порядок работы: исследовать в Postman, зафиксировать в REST-assured.
- Проверки в Postman не ревьюятся как код и не блокируют merge.
- Проверки в REST-assured версионируются, ревьюятся и гоняются в CI.



МОДУЛЬ 5 · ЛЕКЦИЯ 9

Практика на СМП

Базовый тест: что именно мы проверяем

> Осмысленный API-тест проверяет три уровня одного факта — транспорт, бизнес-результат и сгенерированные системой значения.

- Тело запроса содержит все одиннадцать обязательных полей, включая `transmissionDateTime`.
- Транспорт: `statusCode(200)`. Бизнес: `status` = `APPROVED`, `responseCode` = `00`.
- Сгенерированные значения проверяем на формат: `rrn` — 12 цифр, `authCode` — 6 символов.
- `processingTimeMs` может быть равен нулю — проверяем `greaterThanOrEqualTo(0)`, а не `greaterThan(0)`.
- Поле `mti` в ответе намеренно не проверяем — причина на слайде 23.

Код базового теста: given / when / then на практике

> Каждая проверка в блоке `then()` — отдельная строка, и по списку строк видно, что именно проверяется.

```
@Test
void shouldApproveValidTransaction() {
    String requestBody = """
    {
        "mti": "0100",
        "stan": "000001",
        "pan": "4000001234560001",
        "processingCode": "000000",
        "amount": 150000,
        "currencyCode": "643",
        "transmissionDateTime": "2026-06-01T10:30:00Z",
        "terminalId": "TERM001",
        "terminalType": "POS",
        "merchantId": "MERCH12345678901",
        "mcc": "5411",
        "acquirerId": "ACQ001"
    }
    """;

    given()
        .baseUri("http://localhost:8080")
        .contentType(ContentType.JSON)
        .body(requestBody)
    .when()
        .post("/api/transactions")
    .then()
        .statusCode(200)
        .body("stan", equalTo("000001"))
        .body("status", equalTo("APPROVED"))
        .body("responseCode", equalTo("00"))
        .body("rrn", matchesPattern("\\d{12}"))
        .body("authCode", matchesPattern("[A-Z0-9]{6}"))
        .body("processingTimeMs", greaterThanOrEqualTo(0));
}
```

- `given()` — адрес, формат, тело. `when()` — метод и путь. `then()` — семь проверок.
- `transmissionDateTime` обязателен: без него Gateway вернёт `400`, а не `200`.
- Для генерируемых значений используем `matchesPattern`, а не `equalTo`.

Негатив: проверяем не «что-нибудь невалидное», а каждое правило валидатора

> Каждое правило валидатора — отдельный класс эквивалентности, и на каждый класс нужен свой тест.

- Валидатор Gateway проверяет: `mti` строго `0100`, `pan` ровно 16 цифр, `amount` больше нуля, `currencyCode` 3 символа, `mcc` 4 цифры.
- Плюс обязательность одиннадцати полей; `terminalType` необязателен.
- Ответ на нарушение — `400` с телом `ErrorResponse`: `error`, `message`, `timestamp`, `serviceName`, `retryAfterMs`.
- `serviceName` равен `gateway` — по нему видно, кто отклонил запрос.
- Параметризованный тест покрывает классы эквивалентности PAN за один прогон.

Код параметризованного теста: один метод на класс эквивалентности

`@ParameterizedTest` позволяет покрыть все представители класса невалидных значений одним методом и одним списком данных.

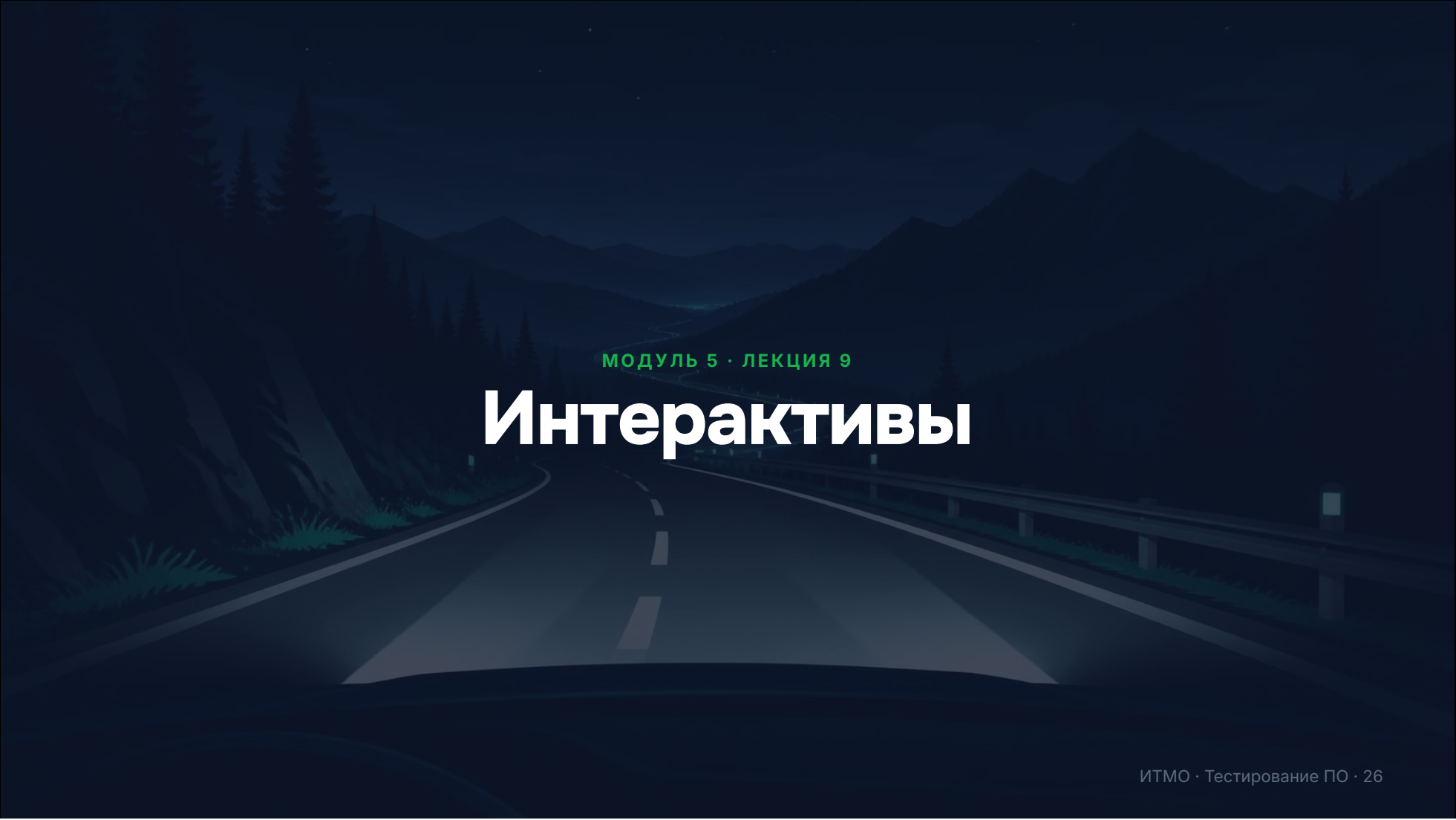
```
@ParameterizedTest
@ValueSource(strings = {"", "123", "400000123456000", "40000012345600001"})
void shouldReturn400ForInvalidPan(String invalidPan) {
    given()
        .baseUri("http://localhost:8080")
        .contentType(ContentType.JSON)
        .body(transactionJson(invalidPan, 150000L))
    .when()
        .post("/api/transactions")
    .then()
        .statusCode(400)
        .body("error", equalTo("VALIDATION_ERROR"))
        .body("message", equalTo("Field 'pan' must be exactly 16 digits"))
        .body("serviceName", equalTo("gateway"))
        .body("retryAfterMs", nullValue());
}
```

- Четыре значения в `@ValueSource` — четыре представителя невалидного класса.
- Проверяем не только статус, но и тело — иначе не отличить нашу ошибку от любой другой `400`.
- `serviceName` привязывает отказ к сервису, который его вынес.

Организация: тест-сьют ломается не от дефектов, а от дублирования и связанности

Поддерживаемость API-тестов определяется тремя решениями — где живут общие настройки, откуда берутся данные и как сгруппированы тесты.

- Базовый адрес и общие заголовки выносятся в `RequestSpecification` или `@BeforeAll`.
- Тестовые данные готовятся в `@BeforeEach`, чтобы тесты не зависели от порядка запуска.
- Тесты группируются по эндпоинтам и сценариям, чтобы запускать точечно.
- Окружение поднимается до прогона: в СМП это `docker compose up -d`.
- Эти же тесты гоняются в лёгком пайплайне CI — как в лекции 6.



МОДУЛЬ 5 · ЛЕКЦИЯ 9

Интерактивы

Интерактив: найдите ошибку в тесте

- Карта создана с балансом 100 рублей, транзакция на 1500 рублей, ожидается отказ по недостатку средств.
- Тест ожидает `statusCode(500)` и статус `DECLINED` с причиной `INSUFFICIENT_FUNDS`.
- Вопрос: что здесь не так и почему это проблема? Код — на следующем слайде.
- Дополнительный вопрос: какие проверки стоит добавить?

Код теста с ошибкой: где именно спрятано неверное ожидание

> Ошибка в ожидании выглядит так же аккуратно, как правильная проверка, — поэтому её и не замечают на code review.

```
@Test
void shouldDeclineForInsufficientFunds() {
    String pan = createCard(10000L);  // баланс 100 рублей

    given()
        .baseUri("http://localhost:8080")
        .contentType(ContentType.JSON)
        .body(transactionJson(pan, 150000L))  // покупка на 1500 рублей
    .when()
        .post("/api/transactions")
    .then()
        .statusCode(500)                      // ← что не так?
        .body("status", equalTo("DECLINED"))
        .body("declineReason", equalTo("INSUFFICIENT_FUNDS"));
}
```

- Код синтаксически безупречен: тест скомпилируется и запустится.

Интерактив: когда спецификация и код расходятся

- Факт первый: спецификация обещает в ответе `mti` со значением `0110`, код возвращает `0100`.
- Факт второй: в поиске транзакций есть фильтры `issuerId` и `mcc`, которых нет в спецификации.
- Факт третий: фильтра `stan` нет ни в спецификации, ни в коде — а тест на него рассчитывает.
- Вопрос: что делать с тестом, который падает из-за первого факта, и кто прав?

Выводы и анонс

> API-тест ценен тогда, когда проверяет контракт целиком — транспорт, данные, бизнес-результат и заголовки.

- REST API — ресурсы, адресуемые URL, с операциями, выраженными HTTP-методами.
- Статус-код — часть контракта; `4xx` и `5xx` различают вину клиента и сервера.
- Заголовки несут часть контракта, которой нет в теле: кеш, повторы, формат файла.
- OpenAPI превращает контракт в машиночитаемый артефакт, а схема даёт дешёвую проверку формата.
- REST-assured — для регрессии, Postman — для разведки; критерий выбора — срок жизни проверки.
- Лекция 10: паттерны — сценарии, доступ, идемпотентность, асинхронные контракты.

Что унести с лекции

- REST — ресурсы, адресуемые URL, с операциями через HTTP-методы
- Статус-код — часть контракта: `4xx` и `5xx` различают вину клиента и сервера
- Заголовки несут часть контракта: кеш, повторы, формат файла
- OpenAPI превращает договорённость в машиночитаемый артефакт
- Схема ловит нарушение формата, consumer-driven контракт — ломающее изменение
- REST-assured для регрессии, Postman для разведки: критерий — срок жизни проверки
- Лекция 10: паттерны — сценарии, доступ, идемпотентность, асинхронность

Источники

- **Spillner, A., Linz, T., & Schaefer, H.** (2014). *Software Testing Foundations* (4th ed.). Rocky Nook. ISBN 978-1-937538-42-2. — Глава 2.4 (принципы тестирования), 3.3 (интеграционное тестирование).
- **Newman, S.** (2021). *Building Microservices* (2nd ed.). O'Reilly. ISBN 978-1-492-03402-5. — Глава 9 (Testing): контракты, consumer-driven contracts, API-тесты.
- **Myers, G., Sandler, C., & Badgett, T.** (2011). *The Art of Software Testing* (3rd ed.). Wiley. ISBN 978-1-118-03196-4. — Глава 4 (Test-Case Design): классы эквивалентности и граничные значения.
- **Fielding, R. T.** (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Диссертация, определившая REST.
- **RFC 9110**: *HTTP Semantics*. https://www.rfc-editor.org/rfc/rfc9110.html — семантика методов, статус-кодов и заголовков.
- OpenAPI Specification: https://spec.openapis.org/oas/latest.html
- JSON Schema: https://json-schema.org/
- Pact — consumer-driven contract testing: https://docs.pact.io/
- REST-assured documentation: https://rest-assured.io/
- Postman documentation: https://learning.postman.com/
- **Спецификация СМП**: `docs/api-spec.md`, `docs/architecture.md` — контракт и карта портов учебного проекта.

Вопросы?

API-тестирование · модуль 5

